ONLINE FOOD ORDERING SYSTEM

C++ Project Report — Code Review and Suggested Improvements

1. Overview

The Online Food Ordering System is a console-based C++ application that displays a fixed menu, allows a user to add food items to a cart, and generates a bill. A 10% discount is applied when the subtotal is at least Rs. 500.

Menu items

ID	Food item
1	Burger
2	Pizza
3	Biryani
4	Sandwich
5	French Fries

2. Class Design

FoodItem class

- Encapsulates the item ID, name, and price using private data members and public getter methods.
- Provides default and parameterized constructors.
- Uses display() and iomanip formatting to present the menu in aligned columns.

Order class

- Manages the cart using a dynamically allocated integer array (quantities), with one quantity per menu item.
- Stores the menu size and maintains a static totalOrders counter for the number of Order objects created.
- Initializes quantities to zero using new int[size](), and releases allocated memory in the destructor.
- Deletes the copy constructor and copy-assignment operator to prevent unsafe shallow copies and double deletion.
- Overloads addItem(): one form uses a default quantity of 1, while another accepts an explicit quantity.

printBill() friend function

The bill-printing function is declared as a friend so it can access Order's private cart data. It calculates the subtotal, applies the discount when eligible, and displays the final amount.

3. OOP Concepts Demonstrated

Concept	How it is used
Encapsulation	Private data members with public getters and controlled access.
Constructor overloading	Default and parameterized constructors in FoodItem.

Concept	How it is used
Function overloading	Multiple addItem() forms for different quantity inputs.
Static member	totalOrders is shared by all Order objects.
Friend function	printBill() accesses private Order data to generate the bill.
Dynamic memory / RAII	Memory is allocated for cart quantities and released by the destructor.

4. Issues and Bugs Identified

Major: reused variable in main()

The variable choice is used both for the main menu selection and for the food item ID. Entering an item ID such as 4 can overwrite the loop-control value and unexpectedly end the loop.

Input validation

If a user enters text where an integer is expected, cin can enter a failed state. Without clearing the error and discarding the invalid input, the program may loop indefinitely.

Menu-size consistency

Functions that index the menu and quantities assume their sizes match. If they do not, an out-of-bounds access may occur.

Order counter meaning

totalOrders counts constructed Order objects, including empty or abandoned orders; it does not necessarily count completed orders.

Hardcoded discount

The discount rule is fixed at 10% for subtotals of Rs. 500 or more.

5. Strengths

- Readable table formatting using left alignment and setw().
- Validation for invalid item numbers and non-positive quantities.
- Deleted copy operations reduce the risk of shallow-copy bugs and double deletion.
- The design provides useful examples of C++ object-oriented programming.

6. Suggested Improvements

- Use a separate int itemId variable when reading a food item ID; keep choice only for the main menu.
- Check cin input state. On failure, call cin.clear() and cin.ignore(...) before retrying.
- Consider std::vector and std::vector to avoid manual new[] and delete[].
- Use std:: prefixes instead of using namespace std; for clearer namespace control.
- Pass or validate the menu size consistently wherever menu and quantity arrays are used.
- Consider adding remove-item and clear-cart options.
- Define MENU_SIZE as a named constant or use a container's size.

7. Example of the Main Bug Fix

Instead of reusing choice to read a food item ID, declare a separate variable:

```
int itemId;
```

```
std::cin >> itemId;
```

Then pass itemId to the function that adds the selected food to the cart. This keeps the main menu choice unchanged and prevents accidental loop termination.

8. Conclusion

The project demonstrates several core C++ OOP concepts, including encapsulation, constructors, function overloading, static members, friend functions, and dynamic memory management. The main priority is to separate menu selection from food item selection and handle invalid input safely. Using standard containers such as std::vector would make the project easier to maintain and safer.